

Laboratorium: Obsługa formularzy w Flask

Aplikacje Internetowe

Wprowadzenie

Obsługa formularzy wymaga zrozumienia metod HTTP. Domyślnie trasy we Flask obsługują tylko metodę `GET`. Aby odebrać dane przesłane przez użytkownika, musimy jawnie zezwolić na metodę `POST`.

Zadanie 1: Prosty formularz kontaktowy

Stwórz widok `kontakt.html`, który pozwoli użytkownikowi wpisać swoje imię oraz treść wiadomości.

Wskazówki:

- W znaczniku `<form>` ustaw atrybut `method="POST"`.
- Każdy element `<input>` musi posiadać unikalny atrybut `name`.

```
1 <form method="POST">
2   <label>Imię:</label>
3   <input type="text" name="imie_uzytkownika">
4   <label>Wiadomość:</label>
5   <textarea name="tresc"></textarea>
6   <button type="submit">Wyślij</button>
7 </form>
```

Listing 1: templates/kontakt.html

Zadanie 2: Odbieranie danych w Pythonie

Zaimplementuj funkcję widoku, która rozpozna czy użytkownik tylko wyświetla stronę (GET), czy przesyła dane (POST).

Wskazówka: Użyj obiektu `request.form` do odczytania wartości z pól formularza.

```
1 from flask import Flask, render_template, request
2
3 @app.route('/kontakt', methods=['GET', 'POST'])
4 def kontakt():
5     if request.method == 'POST':
6         # Pobranie danych po kluczu z atrybutu 'name'
7         imie = request.form.get('imie_uzytkownika')
8         wiadomosc = request.form.get('tresc')
9
10        return f"Witaj {imie}! Twoja wiadomosc: {wiadomosc}"
11
12    return render_template('kontakt.html')
```

Listing 2: Obsługa formularza w app.py

Zadanie 3: Integracja z obiektowością (Projekt)

Wykorzystaj zdobytą wiedzę o klasach (<https://docs.python.org/pl/dev/tutorial/classes.html>). Stwórz klasę `Wiadomosc`, której instancje będą tworzone na podstawie danych z formularza.

Wymagania:

1. Zdefiniuj klasę z metodą `__init__` przyjmującą parametry z formularza.
2. Po przesłaniu formularza, utwórz obiekt tej klasy i wyświetl jego atrybuty w konsoli za pomocą metody `drukuj()`.
3. Wykorzystaj logowanie (`app.logger.info`) do potwierdzenia utworzenia obiektu.

Podsumowanie

- `request.method` – pozwala sprawdzić typ żądania.
- `request.form` – słownik zawierający dane z formularza.
- Pamiętaj: Atrybuty klas w Pythonie są domyślnie publiczne[cite: 8], co ułatwia szybkie przekazywanie danych z formularzy do obiektów.

Zadanie 4: Reprezentacja danych w formacie JSON

Współczesne aplikacje webowe często przesyłają dane w formacie JSON. Twoim zadaniem jest przechwycenie danych z formularza i zwrócenie ich w postaci ustrukturyzowanego obiektu JSON zamiast zwykłego tekstu.

Wskazówki:

- Wykorzystaj funkcję `jsonify` z biblioteki Flask.
- Pamiętaj, że w Pythonie wszystko jest obiektem, a słowniki (`dict`) najłatwiej mapują się na format JSON.
- Możesz spróbować połączyć to z dziedziczeniem (np. Twoja klasa mogłaby dziedziczyć po `dict`, podobnie jak w przykładzie `FileInfo` z wykładu).

```
1 from flask import Flask, request, jsonify
2
3 @app.route('/api/kontakt', methods=['POST'])
4 def api_kontakt():
5     # Pobieranie danych
6     dane = {
7         "imie": request.form.get('imie_uzytkownika'),
8         "wiadomosc": request.form.get('tresc'),
9         "status": "odebrano",
10        "liczba_znakow": len(request.form.get('tresc', ''
11    ))
12    }
13
14    # Logowanie faktu wygenerowania JSONa
```

```

14     app.logger.info(f"Wygenerowano odpowiedź JSON dla: {
    dane['imie']}")
15
16     return jsonify(dane)

```

Listing 3: Przykład zwracania JSON we Flask

Polecenie do zadania:

1. Zmodyfikuj swój formularz tak, aby po wysłaniu danych użytkownik otrzymał odpowiedź w formacie JSON.
2. Dodaj do słownika JSON dodatkowe pole `timestamp` z aktualną datą i godziną (wykorzystaj moduł `datetime`, podobnie jak w przykładach z klasy `FileInfo` [cite: 149, 155]).
3. Sprawdź w narzędziach deweloperskich przeglądarki (zakładka Sieć/Network), czy nagłówek `Content-Type` odpowiedzi to faktycznie `application/json`.

Zadanie 5: Generator CV (Formularz i JSON)

Celem zadania jest stworzenie systemu, który pobiera dane od kandydata-informatyka, wyświetla je w formie eleganckiego szablonu, a następnie zapisuje zgłoszenie do pliku bazy danych `cv_database.json`.

0.1 Krok 1: Struktura Formularza

Stwórz plik `templates/formularz_cv.html`. Formularz powinien zawierać następujące sekcje i pola (atrybut `name`):

- **Dane osobowe:** Imię i nazwisko (`fullname`), Email (`email`), Link do GitHub (`github`).
- **Profil zawodowy:** Stanowisko (np. Python Developer) (`position`).
- **Umiejętności:** Technologie (lista oddzielona przecinkami) (`skills`).
- **Doświadczenie:** Opis ostatnich projektów (`experience`).

0.2 Krok 2: Obsługa logiki i zapis do JSON

W pliku `app.py` stwórz trasę, która odbierze dane. Wykorzystaj standardową bibliotekę `json` do zapisu danych na dysku.

```
1 import json
2 import os
3
4 @app.route('/generator', methods=['GET', 'POST'])
5 def generator():
6     if request.method == 'POST':
7         # 1. Pobranie danych
8         nowe_cv = {
9             "fullname": request.form.get('fullname'),
10            "email": request.form.get('email'),
11            "github": request.form.get('github'),
12            "position": request.form.get('position'),
13            "skills": request.form.get('skills').split(',
14            '), # Zamiana na liste
15            "experience": request.form.get('experience')
16        }
17
18        # 2. Odczyt i dopisanie do pliku JSON
19        file_path = 'cv_database.json'
20        data = []
21        if os.path.exists(file_path):
22            with open(file_path, 'r', encoding='utf-8')
23            as f:
24                data = json.load(f)
25
26        data.append(nowe_cv)
27
28        with open(file_path, 'w', encoding='utf-8') as f:
29            json.dump(data, f, ensure_ascii=False, indent
30            =4)
31
32        return render_template('szablon_cv.html', cv=
33        nowe_cv)
```

```
31 return render_template('formularz_cv.html')
```

Listing 4: Zapis danych z formularza do pliku JSON

0.3 Krok 3: Szablon wyświetlania (Szablon CV)

Stwórz plik `templates/szablon_cv.html`, który wyświetli dane w atrakcyjny sposób.

```
1 <div class="cv-container">
2   <h1>{{ cv.fullname }}</h1>
3   <h2>{{ cv.position }}</h2>
4   <p>Email: {{ cv.email }} | GitHub: <a href="{{ cv.
5     github }}">{{ cv.github }}</a></p>
6
7   <h3>Umiejętności techniczne:</h3>
8   <ul>
9     {% for skill in cv.skills %}
10      <li>{{ skill.strip() }}</li>
11    {% endfor %}
12  </ul>
13
14  <h3>Doświadczenie:</h3>
15  <p>{{ cv.experience }}</p>
16</div>
```

Listing 5: Fragment szablonu Jinja2 dla CV

0.4 Polecenia dla studenta:

1. Zaimplementuj powyższy mechanizm.
2. **Walidacja:** Dodaj sprawdzenie w Pythonie, czy pole `email` zawiera znak "@". Jeśli nie, użyj `app.logger.error` i zwróć komunikat o błędzie.
3. **Rozszerzenie:** Dodaj do pliku JSON pole `id`, które będzie automatycznie inkrementowane przy każdym nowym zgłoszeniu (nawiązanie do atrybutów statycznych klasy).

Fragment kodu

Zadanie 6: Autorska rozbudowa systemu (Wyzwania)

Twoim celem jest udoskonalenie dotychczasowej aplikacji o zaawansowane mechanizmy przetwarzania danych i walidacji. Wybierz i zaimplementuj poniższe funkcjonalności:

1. **Zaawansowana walidacja serwerowa:** Nie polegaj tylko na HTML5. W Pythonie sprawdź, czy:
 - Pole `fullname` zawiera co najmniej dwa wyrazy (imię i nazwisko).
 - Link do GitHub zaczyna się od `https://github.com/`.
 - Jeśli dane są błędne, wyświetl odpowiedni komunikat błędu zamiast zapisywać plik.
2. **Parsowanie umiejętności (Skill Cloud):** Zmień sposób przetwarzania pola `skills`. Użytkownik wpisuje technologie po przecinku, a Ty:
 - Usuń zbędne spacje (`strip`).
 - Zamień wszystkie nazwy na wielkie litery (`uppercase`).
 - Posortuj listę alfabetycznie przed zapisem do JSON.
3. **Podgląd bazy zgłoszeń (Odczyt JSON):** Stwórz nową trasę `/admin/cv`, która odczyta plik `cv_database.json` i wyświetli listę wszystkich kandydatów w formie tabeli. To zadanie wymaga użycia pętli `% for %` w nowym szablonie HTML.
4. **Unikalność zgłoszeń:** Zaimplementuj mechanizm sprawdzający, czy dany adres e-mail istnieje już w pliku JSON. Jeśli tak, zamiast dodawać nowy wpis, zaktualizuj dane istniejącego kandydata (nadpisz stare informacje nowymi).
5. **Eksport do formatu tekstowego (TXT/Markdown):** Dodaj przycisk "Pobierz jako tekst". Po jego kliknięciu serwer powinien wygenerować i przesłać użytkownikowi plik tekstowy z sformatowanym CV (użyj `make_response` z Flask, aby ustawić odpowiednie nagłówki pliku).

6. **Implementacja Klasy Kandydat (Model danych):** Zgodnie z zasadami programowania obiektowego, zamiast operować na surowym słowniku z `request.form`, stwórz klasę `Kandydat`.
- Klasa powinna mieć metodę `to_dict()`, która przygotowuje dane do zapisu w JSON.
 - Dodaj metodę instancji `get_experience_level()`, która na podstawie długości opisu doświadczenia zwraca etykietę: "Junior", "Mid" lub "Senior".